

TJMJ: A Full-Stack Web-Based Taojiang Classic Wang Mahjong Platform for Entertainment & Education

JaiJaiC*

TJMJ Development Team

School of Entertainment Computing, TJMJ Lab

{jaijai.c}@tjmj.dev

2026 年 6 月 24 日

Abstract 桃江经典王麻将 (Taojiang Jingdian Wang Majiang, TJMJ) 是一款面向湖南桃江地区麻将爱好者的全栈网页麻将游戏平台。本项目完整实现了桃江地方麻将的核心规则——包括打筛扳王 (癞子) 机制、扎鸟翻倍、门子加番、组合胡法等复杂计分系统，并提供单人模式、四人联机对战、观战模式三大玩法。技术架构采用 Vue 3 + Vite 前端、Node.js WebSocket 服务端、WebRTC 实时语音通信，并通过 ngrok 实现公网穿透部署。本文系统阐述 TJMJ 的游戏规则数学模型、软件架构设计、核心算法实现（胡牌判定、听牌检测、NPC 策略智能体），并讨论当前技术瓶颈与未来 Mahjong Agent 训练方向。本项目完全开源，仅供家庭娱乐与学术研究使用。

1 Introduction

麻将 (Mahjong) 作为中国传统博弈游戏的代表，在湖南桃江地区形成了独具特色的地方变种——桃江经典王麻将。其核心特征包括：108 张万/筒/条字牌（不含风牌与花牌）、”王”（癞子/万能牌）机制、打筛扳王确定癞子、扎鸟翻倍计分、门子累加番型等复杂规则，与标准国标麻将或四川血战麻将存在显著差异。

目前市面上仅有闲逸等少数 App 提供益阳麻将玩法，但其存在以下不足：(1) 需要注册登录，操作繁琐；(2) 规则与桃江本地玩法不完全一致；(3) 缺乏拖拽手牌、实时语音、文字弹幕等功能；(4) 无法训练 AI 对弈智能体。

为此，我们设计并开发了 TJMJ——一款与桃江麻将规则高度一致、无需安装、点击链接即可进入的网页麻将平台。本文将从游戏规则数学建模（第 2 节）、系统架构（第 3 节）、核心算法（第 4 节）以及现存问题与未来工作（第 5 节）四个方面对该平台进行完整的技术阐述。

*Correspondence: Project homepage: <https://github.com/JaijaiC/TJMJ>

2 Game Rules & Mathematical Formulation

2.1 Tile Set & Terminology

桃江麻将使用精简牌组——共 108 张牌，三种花色各 36 张：

- 万 (Wan, W): 一万 ~ 九万，各 4 张 (11–19)
- 条 (Tiao, T): 一条 ~ 九条，各 4 张 (21–29)
- 筒 (Tong, B): 一筒 ~ 九筒，各 4 张 (31–39)

编码约定：花色十位数 (1= 万, 2= 条, 3= 筒) + 数字个位数。例：38 表示八筒，15 表示五万。

术语表：

- 王 ($Wang$): 癞子/万能牌，可替代任意牌使用。由开局”打筛扳王”确定。
- 地 (Di): 扳开的那张牌本身，用于判定地胡。
- 将 ($Jiang$): 数字为 2、5、8 的牌（任意花色），共 36 张。胡牌必需品。
- 一句话 ($YiJuHua$): 三张牌构成 AAA （刻子）或 ABC （顺子，同花色连续）。
- 听牌 ($Ting$): 13 张手牌差任意一张即可胡牌的状态。
- 扎鸟 ($ZhaNiao$): 胡牌后翻下一张牌，根据数字确定输赢翻倍。
- 门子 ($MenZi$): 特殊胡牌类型（碰碰胡、清一色等），采用累加番规则。

2.2 Dice Rolling & Wall Breaking

开局由庄家掷两枚骰子，骰子点数记为 d_1, d_2 ($d_1 \leq d_2$, $d_i \in \{1, \dots, 6\}$)。设 $S = d_1 + d_2$ 。

定庄与数墙：庄家位置编号为 P_0 ，逆时针依次为下家 P_1 、对家 P_2 、上家 P_3 。从庄家开始数到第 S 家（庄家 = 1, 5, 9），该玩家面前的牌墙即为起手墙。

开牌位置：在该墙右手边开始往左数，跳过 d_1 （较小骰数）叠，从下一叠开始抓牌。

抓牌顺序：庄家起抓，每人每次取一码（两叠 = 4 张），共三轮 ($3 \times 4 = 12$ 张/人)。然后”收庄”每人各补 1 张（各得 13 张），庄家再补 1 张（14 张），游戏开始。

扳王规则：从起手墙对应玩家的下家墙左手边开始往右数，到第 d_2 （较大骰数）叠处扳开最上面一张牌，此牌即为”地” (T_d)， $T_d + 1$ （模 10 循环，花色不变）即为”王”/癞子：

$$T_w = \begin{cases} \lfloor T_d/10 \rfloor \times 10 + 1, & \text{if } T_d \bmod 10 = 9 \\ T_d + 1, & \text{otherwise} \end{cases} \quad (1)$$

2.3 Basic Actions & Priority

玩家回合可执行以下操作，优先级由高到低：

1. 胡 (Hu) 一点炮/自摸，直接获胜（最高优先级）
2. 碰 ($Pong$) / 杠 ($Gang$) 一三张相同牌碰之；四张相同可开杠
3. 吃 (Chi) 一仅能吃上家打出的牌，构成顺子

优先级规则：点炮（胡）> 碰（杠）> 吃。抢杠规则：若明杠的牌恰好是别家要胡的牌，直接判杠家全赔（相当于抓炮）。

吃 (Chi)

只能吃上家打出的牌，且必须与手中两张牌构成同花色顺子。例：上家打 4 万，手中有 (2,3) 或 (3,5) 或 (5,6) 万即可吃。

碰 (Pong)

任意玩家打出一张牌，若手中有两张相同即可碰。碰后三张亮出（副露），手牌数减 2。

杠 (Gang)

分明杠（他人打出，手中有三张）和暗杠（手中四张，未亮出）。开杠后重新掷骰子，从牌墙末尾倒数摸三张。若摸到能胡的牌直接胡（杠上花），否则由下家开始依次判定能否胡此三张。

2.4 Winning Patterns (Hu Types)

胡牌的基本结构为：一对将 (Jiang) + 四句话 (Sentence)，其中”一句话”可以是刻子 (AAA) 或顺子 (ABC)。

表 1 列出了所有胡法类型及其番数。门子胡法采用累加番规则：胡牌总番数 = 基础胡法番 + 各门子番数之和。

表 1: 胡法类型与番数对照表

类型	条件	番数
平胡	有王，基本构成	1 番
硬庄	无王，基本构成	2 番
碰碰胡	四句话全为刻子 AAA	2 番
清一色	手牌全部同花色	2 番
七小对	14 张组成 7 个对子	2 番
将将胡	14 张全部为 2/5/8	2 番
起手胡	庄家 14 张首轮直接胡	2 番
开杠	明杠/暗杠后摸三张胡牌	2 番
地胡	摸到三张”地”即可胡	2 番
天胡	手牌有 3 个王	3 番
天天胡	手牌有 4 个王	5 番
黑天胡	首轮无王、无将、无一句话	2 番
地胡带拖	三张地不胡，当一句话用	3 番

组合胡法

当多种门子同时满足时，番数累乘。通用计算公式：

$$S_{\text{自摸}} = \left(\sum \text{门子倍数} \right) \times F_{\text{扎鸟}} \times F_{\text{杠}} \quad (2)$$

实例：

- 天胡七小对全中： $(3 + 2) \times 2 = 10$ 番

- 天胡杠上花全中： $3 \times 2 \times 2 = 12$ 番
- 硬庄将将胡碰碰胡天胡杠上花全中： $(2 + 2 + 2 + 2) \times 2 \times 2 = 32$ 番

2.5 Scoring & ZhaNiao (扎鸟)

胡牌分为自摸（自己摸到胡牌）和抓炮（他人打出胡牌）两种方式。

自摸计分

胡牌后进行扎鸟：翻开本该下家摸的那张牌 (T_z)，根据 $T_z \bmod 10$ 的值判定翻倍对象：

- 1, 5, 9（全中）：三家输分均翻倍
- 2, 6：下家输分翻倍
- 3, 7：对家输分翻倍
- 4, 8：上家输分翻倍

设基础分 $B = 3$ （平胡 1 番 $\times$ 底分 3），番数 M ，扎鸟系数 $Z \in \{1, 2\}$ ：赢家得分 = 输家各支付 $B \times M \times Z$ （扎中者）或 $B \times M$ （未扎中者）。

核心规则

有癞子只能自摸（不能抓炮），无癞子可以抓炮。这一规则使”硬庄”（无王胡牌）更具策略价值。

3 System Architecture

3.1 Overview

TJMJ 采用典型的三层 B/S 架构，整体部署拓扑如图 1 所示。

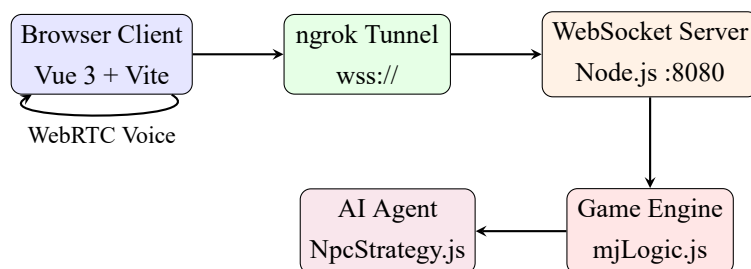


图 1: 系统架构拓扑图

3.2 Frontend —Vue 3 SPA

前端是基于 Vue 3 的单页应用（SPA），使用 Vite 构建，部署于 GitHub Pages。

技术栈：

- **Vue 3.5** (Composition API + reactive) —响应式 UI 框架
- **Vite 8.x** —构建工具 (HMR 热更新)
- **Web Speech API** —浏览器内置语音播报

- **Web Audio API** —BGM 背景音乐管理
- **WebRTC** 点对点实时语音通信

源码结构:

```

1 frontend/src/
2   main.js           # Vue 入口
3   App.vue          # 主游戏界面 (3000+ loc)
4   style.css        # 全局样式
5   core/
6     constants.js    # 牌花色定义、258判定、王计算
7     HuCalculator.js  # 胡牌判定引擎
8     RuleChecker.js  # 吃碰杠听牌检测
9   utils/
10    mjLogic.js       # 洗牌、牌墙布局
11    speech.js        # 语音播报(吃碰杠胡)
12  ai/
13    NpcStrategy.js   # NPC 出牌决策策略
14  network/
15    client.js        # WebSocket 联机客户端

```

3.3 Backend —Node.js WebSocket Server

服务端基于 Node.js + ws 库，负责房间管理、游戏状态同步、操作合法性校验。

技术栈:

- **Node.js** —运行时
- **ws (v8.18)** —WebSocket 服务端
- **uuid (v10)** —房间号生成

源码结构:

```

1 server/
2   index.js          # WS 入口, 消息路由 (180 loc)
3   GameRoom.js       # 房间管理、游戏流程
4   game/
5     constants.js     # 牌定义 (前端镜像)
6     mjLogic.js       # 洗牌、牌墙布局 (前端镜像)
7     HuCalculator.js  # 胡牌计分引擎
8     RuleChecker.js   # 操作合法性校验

```

3.4 Communication Protocol

客户端与服务器通过 WebSocket 进行 JSON 消息通信。核心消息类型:

表 2: WebSocket 消息协议

type	方向	说明
create_room	C→S	创建房间
join_room	C→S	加入房间 (8888= 测试)
ready	C→S	准备就绪
discard	C→S	出牌
draw	C→S	摸牌
action	C→S	吃/碰/杠/胡操作
pass	C→S	过牌
pay	C→S	手动转分
emoji	C↔S	表情
chat	C↔S	文字弹幕
webrtc_*	C↔S	WebRTC 信令中继
join_spectate	C→S	观战

3.5 Network Topology & ngrok

本地开发使用 `ws://localhost:8080` 直连；公网部署通过 ngrok 建立 TCP 隧道，客户端自动检测 `location.hostname` 切换 `wss://` 连接。消息发送采用缓冲队列机制：连接建立前的消息先入队，`onopen` 后批量冲刷，避免连接尚未就绪时 `send` 报错。

4 Core Algorithms

4.1 Hu Detection Algorithm (胡牌判定)

胡牌判定是麻将引擎最核心的算法模块，对应文件 `HuCalculator.js`。算法采用递归回溯 + 贪心剪枝策略：

1. 预处理：分离王牌与普通牌，统计 n_w （王数量）和 n_d （地数量）
2. 特殊拦截：首轮 14 张检查黑天胡（无王、无将、无一句话）
3. 顶级判定：4 王 $\rightarrow$ 天天胡, 3 王 $\rightarrow$ 天胡, 3 地 $\rightarrow$ 地胡
4. 常规判定：调用 `checkNormalHu()` 递归去除刻子/顺子
5. 七小对判定：调用 `check7Pairs()` 贪心配对
6. 门子判定：将将胡 $\rightarrow$ 清一色 $\rightarrow$ 碰碰胡 $\rightarrow$ 硬庄 $\rightarrow$ 软庄，依序匹配

`checkNormalHu(normalTiles, wangCount)` 的递归逻辑：

1. 若王充足，补齐对子（将）后递归去除句子
2. 每次选择最小的牌 $t_{\min}$ ，尝试三种消除：
 - 刻子： $\{t, t, t\}$ 三张相同
 - 顺子： $\{t, t+1, t+2\}$ 三张连续（同花色）
 - 对子： $\{t, t\}$ 作为将（仅当将未确定时）
3. 若普通牌无法匹配，消耗 1 个王替代，继续递归
4. 全部消除成功 $\rightarrow$ 胡牌；任何分支失败 $\rightarrow$ 回溯

4.2 Ting Detection Algorithm (听牌检测)

听牌检测用于判断 13 张手牌是否差一张即可胡牌，以及开杠后是否能听牌。对应 `RuleChecker.isTing()`：

1. 手牌数若不为 $3n+1$ ，直接返回 `false`
2. 枚举所有可能的 34 种牌（效率优化：仅搜索手牌已有花色 ± 2 范围）
3. 对每种候选牌 t ，模拟加入手牌后调用 `HuCalculator.checkHu()`
4. 若任意候选能胡 $\rightarrow$ 听牌 `true`；否则 `false`

此算法的正确性依赖于胡牌判定引擎的完备性。计算复杂度 $O(34 \times 3^k)$ ，其中 k 为递归深度（通常 $k \leq 4$ ，实战 $< 1\text{ms}$ ）。

4.3 NPC Strategy Agent (AI 出牌决策)

`NpcStrategy.js` 实现了一个基于启发式规则的单人模式 NPC 智能体。当前版本采用手牌价值评估 + 听牌最大化策略：

1. **听牌优先**: 遍历每张手牌, 模拟丢弃后检查是否听牌 (`findBestDiscardToTing()`)。若能直接听牌, 优先打出该牌。
2. **价值排序**: 若未听牌, 按优先级评估手牌价值 (`chooseWorstTile()`):
 - 孤张 (无相邻牌、非将、非王) → 最优先丢弃
 - 边张/坎张 (只能等 1 张进张) → 次优先
 - 搭子 (差 1 张成顺子/刻子) → 保留
 - 将对 (2/5/8 对子) → 高价值保留
 - 王牌 → 绝不丢弃
3. **安全出牌**: 参考弃牌池, 避免打出别家可能胡的”炮牌”

未来工作: 将 NPC 升级为基于深度强化学习 (DRL) 的 Mahjong Agent, 使用蒙特卡洛树搜索 (MCTS) 与自我对弈 (Self-Play) 训练。

5 Features & User Interface

5.1 Game Modes

平台提供三种游戏模式:

1. **单人模式**: 玩家 vs 3 个 NPC AI, 适合练习
2. **联机模式**: 4 人实时对战, 创建/加入房间 (4 位数字房间号)
3. **观战模式**: 旁观正在进行的对局 (每次仅可选择一方视角)

5.2 Interactive Features

- **拖拽手牌**: 手牌可自由拖动排列, 自动摸张固定在右侧
- **表情系统**: 点击头像可向其他玩家发送表情包
- **实时转分**: 点击头像下方计分区域可手动转分
- **文字弹幕**: 右上角聊天按钮, 支持全屏弹幕聊天
- **实时语音**: 基于 WebRTC 的点对点语音通信
- **BGM 控制**: 首页轻触声音按钮开关背景音乐; 局内右上角切换钢琴纯音乐模式
- **语音播报**: 浏览器内置 TTS 播报”吃””碰””杠””胡了”等操作

6 Discussion

6.1 Known Issues

当前版本存在以下技术瓶颈:

1. **兼容性问题**: 安卓/iOS、手机/桌面端渲染差异导致偶发 UI 错位
2. **实时语音不稳定**: WebRTC 多人 (≥ 3) 出现断连/回声, 需引入 SFU 中转
3. **结算界面不完整**: 结束时无法查看赢家完整手牌
4. **麻将音效缺失**: 需补充”一万””吃””我胡了”等地方语音音效
5. **昵称不可修改**: 当前无改名功能
6. **断线重连缺失**: 服务器断开后无法恢复房间状态

7. 复杂吃碰逻辑卡死：高频操作下偶发 UI 冻结，需引入操作节流与状态机
8. 托管模式：胡牌判定失败或卡死后缺乏自动恢复机制

6.2 Future Work: Mahjong Agent

- 项目规划的核心远期目标是训练一个高性能 Mahjong Agent:
- 利用完美信息自我对弈 (Self-Play) 生成训练数据

• 采用深度 Q 网络 (DQN) 或 PPO 算法学习出牌策略

• 引入蒙特卡洛树搜索 (MCTS) 提升决策质量

• 目标: Agent 达到或超越人类高手水平，成为玩家的学习陪练

7 Methods

7.1 Development Environment

表 3: 开发环境与依赖

组件	版本/技术
前端框架	Vue 3.5 (Composition API)
构建工具	Vite 8.0
服务端	Node.js + ws 8.18
语音	Web Speech API + WebRTC
内网穿透	ngrok
版本管理	Git + GitHub
部署	GitHub Pages (前端) + ngrok (后端)
Node 依赖	vue, vite, @vitejs/plugin-vue, gh-pages ws, uuid

7.2 Deployment

启动命令 (Windows):

```
1 # 一键启动所有服务
2 .\start-all.bat
3 # 依次启动: Server :8080 -> Frontend :5173 -> ngrok
```

手动启动:

```
1 # Terminal 1 - Game Server
2 cd server && npm run dev      # WebSocket :8080
3
4 # Terminal 2 - Frontend
5 cd frontend && npm run dev    # Vite :5173
6
7 # Terminal 3 - ngrok (optional)
```

```
8 ngrok http 8080
```

7.3 Testing

测试房间 8888 常驻运行，自动在状态异常时重建。满 4 人且全部 ready 后 0.8 秒自动开局（create_room 满员则 1.5 秒延迟开局）。

Code Availability

本项目完全开源，代码仓库：<https://github.com/JaijaiC/TJMJ>

欢迎 Fork / Star / Pull Request。

Disclaimer

本项目为个人娱乐学习用途，不涉及任何商业行为。所有规则及实现仅供参考，实际游玩请以当地线下规则为准。因使用本软件产生的任何争议或损失，开发者不承担相关责任。

Acknowledgments

感谢湖南桃江的家人朋友提供规则校验与测试反馈；感谢开源社区 (Vue.js, Node.js, Vite) 提供的优秀工具链。

参考项目：

- OpenMajiang (<https://gitee.com/mirrors/OpenMajiang>)
- 武汉麻将在线 (<http://cdn0001.afrxvk.cn/whmj/go.html>)